

Data Transfer Programs

- 1) **Write an ALP to move a block of n bytes of data from source (20h) to destination (40h) using internal RAM.**

```
ORG 0000H      ; Initialize the memory
MOV R2,#05H    ; Load R2 (counter) with 05H

MOV R0,#20H    ; Load R0 with 20H (starting address of source block)

MOV R1,#40H    ; Load R1 with 40H (starting address of destination block)
LOOP: MOV A,@R0 ; Copy the contents of memory location pointed by R0 into acc

MOV @R1,A      ; Copy the contents of acc into memory location pointed by R1

INC R0         ; Increment R0
INC R1         ; Increment R1
DJNZ R2, LOOP  ; Decrement R2 and jump to LOOP if R2≠0

END            ; End of the program
```

- 2) **Write an ALP to move a block of n bytes of data from source (2000h) to destination (2050h) using external RAM.**

```
ORG 0000H      ; Initialize the memory
MOV DPTR,# 2000H ; load dptr with source location address
MOV R0, #50H    ; r0 is loaded with destination location lower byte
MOV R1, #20H    ; r1 is loaded with destination location upper byte
MOV R2, #05H    ; r2 is loaded with block size
BACK: MOVX A, @DPTR ; acc is loaded with content of address present in dptr
      PUSH DPH    ; save dptr content into stack
      PUSH DPL
      MOV DPH, R1  ; load dph with R1 destination location upper byte
      MOV DPL, R0  ; load dpl with R0 destination location lower byte

      MOVX @DPTR, A ; mov accumulator content to destination location
      INC DPTR     ; increment destination location
      MOV R1,DPH   ; mov dph content to R1
      MOV R0,DPL   ; mov dpl content to R0
      POP DPL      ; retrieve the content from stack
      POP DPH
      INC DPTR     ; increment source location
      DJNZ R2, BACK ; decrement R2 , if not zero jump to label 'BACK'
      END         ; End of the program
```

- 3) Write an ALP to exchange the source block starting with address 20h (Internal RAM) containing N (05) bytes of data with destination block starting with address 40h (Internal RAM).

```

ORG 0000H           ; Initialize the memory
MOV R0,#20H         ; Load R0 with starting address of first location
MOV R1, #40H        ; Load R1 with starting address of second location
MOV R2, #05H        ; Load R2 with counter
UP:  MOV A, @R0       ; Copy the contents of memory location pointed by R0 in acc
     XCH A, @R1       ; Exchange A with content of address present in R1 register
     MOV @R0, A       ; Copy the content of A into memory location pointed by R0
     INC R0           ; Increment R0
     INC R1           ; Increment R1
     DJNZ R2, UP      ; Decrement R2, if not zero jump to label UP
     END             ; End of the program

```

- 4) Write an ALP to exchange the source block starting with address 10h (Internal memory) containing N (6) bytes of data with destination block starting at location 2000h (External memory).

```

ORG 000H           ; Initialize the memory
MOV R0, #10H       ; Load R0 with source location address
MOV DPTR, #2000H   ; Load dptr register to hold destination address
MOV R2, #06H       ; Load R2 with block size
BACK: MOVX A, @DPTR ; Load acc with the content of address present in dptr
     MOV B, @R0     ; Load register B with internal memory content
     XCH A,B        ; Exchange acc and register B contents
     MOVX @DPTR, A  ; Copy the content of acc to dptr address (external)
     MOV @R0,B      ; Copy the content of register B to R0 address (in
     INC R0         ; Increment register R0
     INC DPTR       ; External memory location incremented by 1
     DJNZ R2, BACK  ; Decrement register R2 if not zero jump to label back
     END           ; End of the program

```